# Tabla de contenidos


1. Funciones Generadoras
    a. Ejemplos
    b. Otra forma de hacer iterable una estructura propia
    c. También podemos interactuar con la función generadora enviando mensajes
    d. ¿Qué hace que una función sea generadora en Python?


# Funciones Generadoras

Como vimos anteriormente, los **generadores** nos permiten iterar sobre secuencias de datos sin la necesidad de almacenarlos en alguna estructura especial, evitando el uso innecesario de memoria. Una vez que terminamos de iterar sobre un generador, el generador desaparece.

En este contexto, las funciones en Python también tienen la posibilidad de funcionar como generadores, con la sentencia `yield`. El *statement* `yield` es un análogo a `return`, con ciertas diferencias. Por un lado, `yield` se encarga de retornar el valor indicado, pero también se asegura que en la próxima llamada a la función, la ejecución parta desde donde se dejó antes.

En otras palabras, trabajamos con una función que una vez que entrega un valor a través de `yield`, está cediendo el control sólo en forma temporal, asumiendo que pronto será utilizada nuevamente para generar más valores.

Creemos nuestra primera función generadora de números decrecientes:

```
from typing import Generator
def conteo_decreciente(n: int) -> Generator:
    print(f"Contando en forma decreciente desde {n}")
    while n > 0:
        yield n
        n -= 1
```

Vemos que cuando se llama a la función generadora, esta no ejecuta nada:

```
x = conteo_decreciente(4)
```

Esto se debe a que cuando se invoca la función generadora, esta retorna un **generador**. Luego, `x` es un generador de números desde el 10 hasta el 1.

```
print(type(x))
```

```
<class 'generator'>
```

Podemos usar el generador directamente en un `for`, ya que como vimos estos implementan `__iter__` devolviendo `self`.

```
for number in x:
    print(number)

Contando en forma decreciente desde 4
4
3
2
1
```

También se puede usar `next`:

```
x = conteo_decreciente(5)
print(next(x))

Contando en forma decreciente desde 5
5
```

# Ejemplos

Veamos un ejemplo de una función generadora de números de Fibonacci:

```
from typing import Generator


def fibonacci() -> Generator:
    a, b = 0, 1
    while True:  # Notar que este generador nunca "se agota"
        yield a
        a, b = b, a + b


generador_fibonacci = fibonacci()

# Imprimimos los primeros 5 elementos
for i in range(5):
    print(next(generador_fibonacci))

0
1
1
2
3
```

---

Además, veamos cómo utilizar las funciones generadoras para leer cargar los datos de un archivo, sin cargar todos los datos en memoria.

```
from os import path
from typing import Generator


def cargar_archivo(path: str) -> Generator:
    """Lee el contenido de un archivo, sin cargar toda la información
del mismo en memoria."""

    with open(path, encoding='utf-8') as file:
        for line in file:
            yield line


lenguas_extintas = cargar_archivo(path.join('data',
'lenguas_extintas.csv'))

for i in range(5):
    print(next(lenguas_extintas))
```

```
id,Name in English,Countries,Country codes,Degree of
endangerment,Number of speakers

0,South Italian,Italy,ITA,Vulnerable,75000000

1,Sicilian,Italy,ITA,Vulnerable,50000000

2,Low Saxon,Germany;Denmark;Netherlands;Poland;Russian
Federation,DEU;DNK;NLD;POL;RUS,Vulnerable,48000000

3,Belarusian,Belarus;Latvia;Lithuania;Poland;Russian
Federation;Ukraine,BRB;LVA;LTU;POL;RUS;UKR,Vulnerable,40000000
```

Es importante notar, que al momento de leer el archivo estamos iterando sobre el mismo (`for line in file`), lo que en sí, nos entrega un generador que itera sobre las líneas del archivos.

En cambio, al usar la función `file.readlines()`, esto nos retornar una lista con todos los datos del archivo; por lo que al hacer esto, estamos cargando toda la información del archivo en la memoria del computador.

---

También veamos un ejemplo de que las funciones generadoras pueden operar con otras colecciones, como listas:

```
from typing import Generator, List


def maximo_acumulativo(valores: List[int]) -> Generator:
    """Retorna el máximo visto hasta ahora en una colección de
valores."""
```

```
    max_ = float('-inf')
    for valor in valores:
        max_ = max(valor, max_)
        yield max_


lista = [1, 10, 14, 7, 9, 12, 19, 33]

for i in maximo_acumulativo(lista):
    print(i)
```

```
1
10
14
14
14
14
19
33
```

## Otra forma de hacer iterable una estructura propia

Si tenemos una estructura de datos propia, podemos usar una función generadora en `__iter__` en vez de crear nuestra propia clase iteradora. Para ilustrarlo usaremos el mismo ejemplo del principio:

```
from typing import Generator


class IterableListaNumeros:
    def __init__(self, objeto: list) -> None:
        self.objeto = objeto.copy()

    def __iter__(self) -> Generator:
        while self.objeto:
            yield self.objeto.pop(0)

datos = [1, 2, 3, 4, 5]
iterable = IterableListaNumeros(datos)

for x in iterable:
    print(x, end=" ")
```

```
1 2 3 4 5
```

```
iterador = iter(iterable)
print(type(iterador))
```

```
<class 'generator'>
```

# También podemos interactuar con la función generadora enviando mensajes

El método `send()` permite enviar un valor hacia el generador, lo que significa que la expresión `yield` lo recibirá. El valor enviado puede ser usado para asignarlo a otra variable, por ejemplo `v = yield value` guardará en la variable `v` el valor enviado con `send()`.

Veamos un ejemplo de una función generadora que entrega números que se incrementan según lo que le es enviado mediante `send`:

```
from typing import Generator


def funcion_generadora_send() -> Generator:
    contador = 0

    while True:
        valor_recibido = yield contador
        print("Hemos recibido {}".format(valor_recibido))

        if valor_recibido is None:  # Consideraremos 0 si nos llega un
None
            valor_recibido = 0

        print("Sumaremos {} a nuestro
contador".format(valor_recibido))
        contador += valor_recibido  # Sumamos el valor recibido al
contador que llevamos

generador_send = funcion_generadora_send()
```

Lo primero en lo que nos tenemos que fijar es que debemos avanzar hasta `yield` antes de poder enviar valores. Es decir, la primera vez no podremos enviar nada, sólo usar `next`:

```
try:
    generador_send.send(5)
except TypeError as error:
    print(TypeError.__name__+':', error)
```

```
TypeError: can't send non-None value to a just-started generator
```

Ya hecho el primer `yield`, podemos enviarle valores a la función generadora:

```
next(generador_send)
```

```
0
```

```
generador_send.send(5)
```

```
Hemos recibido 5
Sumaremos 5 a nuestro contador

5
```

### (Pequeño paréntesis)

Acá podras preguntarte ¿Por qué se guarda el valor `5` si es que ya fue ejecutada la linea `valor_recibido = yield contador` al ejecutar `next(generador_send)`? Es una pregunta muy válida, y para esto es necesario entender cómo Python maneja estos comandos para obtener la respuesta.

Esto se debe a que, aunque la línea `valor_recibido = yield contador` ya se ejecutó en cuanto al `yield`, la asignación de `valor_recibido` no ocurre hasta que el generador se reanuda con un `send()`. Por eso, cuando se ejecuta `generador_send.send(5)`, el valor `5` es recibido y asignado a `valor_recibido`

En otras palabras, la clave está en que el `yield` actúa como una puerta de entrada para los valores enviados. Al pausar la ejecución, se "espera" a que se le envíe un valor, el cual se asignará a la variable cuando la ejecución se reanude. Esto permite una **comunicación bidireccional** donde, además de emitir valores, el generador puede recibir información dinámica a través de `send()`.

### Continuamos...

Luego, hacer `next`, es equivalente a hacer `send` de `None`.

```
next(generador_send)

Hemos recibido None
Sumaremos 0 a nuestro contador

5

generador_send.send(10)

Hemos recibido 10
Sumaremos 10 a nuestro contador

15
```

## Funciones Generadoras a partir de un Iterable

Otra manera de crear una función generadora es utilizando el *statement* `yelid from`. Éste nos permite **crear un generador a partir de algún otro iterable** sin la necesidad de ocupar algún tipo de bucle o *loop* (*for* o *while*) para recorrer sus elementos. Debido a sus caractetísticas, este *statement* nos permite escribir código más limpio, modular y fácil de mantener cuando se trabaja con **generadores compuestos o anidados**.

Veamos un ejemplo de una función generadora creada a partir de un iterable utilizando el *statement* `yield from`:

```
def generador_simple():
    yield from range(5)

for valor in generador_simple():
    print(valor)
```

```
0
1
2
3
4
```

Además como un generador es un iterable, podemos **crear un generador a partir de otro generador**

```
def generador_hijo():
    for i in range(5):
        yield i

def generador_padre():
    yield from generador_hijo()


for valor in generador_padre():
    print(valor)
```

```
0
1
2
3
4
```

# ¿Qué hace que una función sea generadora en Python?

Como comentábamos en un inicio, la funciones se vuelven funciones generadoras por medio de la sentencia `yield`, pero ¿toda función que contenga `yield` es una función generadora? ¿necesitamos que el `yield` se ejecute para que una función sea considerada generadora?

A continuación, definamos distintas funciones y veamos si Python las considera funciones generadoras o no:

```
def funcion1():
    '''Función con el yield dentro del scope principal.'''
    yield 1


print(type(funcion1()))
```

```
<class 'generator'>
```

```
def funcion2():
    '''
    Función con el yield dentro de un if que es falso,
    donde además la función presenta un return.
    '''
    if False:
        yield 1
    return 1


print(type(funcion2()))
```

```
<class 'generator'>
```

```
def funcion3():
    '''
    Función que contiene un yield,
    pero que nunca alcanza a ser ejecutado.
    '''
    return 1
    yield 1


print(type(funcion3()))
```

```
<class 'generator'>
```

Como podemos observar, para que una función sea considerada generadora basta con que esta **contenga un `yield`**, independientemente de si este alcanza a ser ejecutado o no.